Lab Assignment

Name : Mohon Kumer
ID : 2110976127
Session : 2020 - 21

Course code : CSE3242
Course title : Operating Systems Lab

Answer - 1:

Problem: Write a C program to create a main process named ‘parent_process’ having 3 child processes without any grandchildren processes. Trace parent and child processes in the process tree. Show that child processes are doing addition, subtraction and multiplication on two variables initialized in the parent_process

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int a = 10, b = 5;  // Variables initialized in the parent
process
    pid_t pid1, pid2, pid3;

    printf("Parent Process (PID: %d): Starting with values a = %d, b
=
 %d\n", getpid(), a, b);

    // First child - addition
    pid1 = fork();
    if (pid1 == 0) {
        printf("Child 1 (PID: %d, PPID: %d): Doing addition: %d + %d
= %d\n", getpid(), getppid(), a, b, a + b);
        exit(EXIT_SUCCESS);  // Child exits
    }

    // Second child - subtraction
    pid2 = fork();
    if (pid2 == 0) {
        printf("Child 2 (PID: %d, PPID: %d): Doing subtraction: %d -
%d = %d\n", getpid(), getppid(), a, b, a - b);
        exit(EXIT_SUCCESS);  // Child exits
    }

    // Third child - multiplication
    pid3 = fork();
    if (pid3 == 0) {
```

```
        printf("Child 3 (PID: %d, PPID: %d): Doing multiplication: %d
* %d = %d\n", getpid(), getppid(), a, b, a * b);
        exit(EXIT_SUCCESS);  // Child exits
    }

    // Parent process waits for all children
    wait(NULL);
    wait(NULL);
    wait(NULL);

    printf("Parent Process (PID: %d): All child processes
completed.\n", getpid());

    exit(EXIT_SUCCESS);
}
```

[1] [5]

Output:

```
systemd──2*[agetty]
        ├─check-new-relea──{check-new-relea}
        ├─cron
        ├─dbus-daemon
        ├─init-systemd(Ub─┬─SessionLeader──Relay(470)──bash──problem1──3*[problem1]
                          ├─SessionLeader──Relay(941)──bash──pstree
                          ├─init──6*[{init}]
                          ├─login──bash
                          └─{init-systemd(Ub}
        ├─polkitd──3*[{polkitd}]
```

Explanation:

The output shows three child processes performing addition, subtraction, and multiplication using two numbers (a = 10, b = 5) set by the parent. Each child prints its own process ID (PID) and its parent's ID (PPID), proving that they’re direct children of the main process. No grandchildren are created, and each task runs independently. This confirms a flat process tree structure with proper parent-child relationships.

Answer - 2:

Problem: Write a C program to create an orphan process.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sched.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        // Fork failed
        perror("fork failed");
        exit(EXIT_FAILURE);
    } else if (pid > 0) {
        // Parent process
        printf("Parent process:\n");
        printf("My PID: %d\n", getpid());
        printf("My Parent PID: %d\n", getppid());
        sleep(20);
        printf("Parent process exiting...\n");
        exit(EXIT_SUCCESS);
    } else {
        // Child process
        printf("Child process (before orphan):\n");
        printf("My PID: %d\n", getpid());
        printf("My Parent PID: %d\n", getppid());
        sleep(50);  // Ensure parent exits before child runs this
        printf("Child process (after orphan):\n");
        printf("My PID: %d\n", getpid());
        printf("My Parent PID: %d\n", getppid());
    }

    exit(EXIT_SUCCESS);
}
```

[1], [5]

Output:

```
│   │   │   ├─gnome-shell-cal(2677)─┬─{gnome-shell-cal}(2678)
│   │   │   │                      ├─{gnome-shell-cal}(2679)
│   │   │   │                      ├─{gnome-shell-cal}(2681)
│   │   │   │                      ├─{gnome-shell-cal}(2682)
│   │   │   │                      ├─{gnome-shell-cal}(2683)
│   │   │   │                      └─{gnome-shell-cal}(2966)
│   │   │   ├─gnome-terminal-(3605)─┬─bash(3612)───pstree(10151)
│   │   │   │                      ├─bash(3729)───problem2(10148)───problem2(10149)
│   │   │   │                      ├─{gnome-terminal-}(3606)
│   │   │   │                      ├─{gnome-terminal-}(3607)
│   │   │   │                      ├─{gnome-terminal-}(3609)
│   │   │   │                      ├─{gnome-terminal-}(3610)
│   │   │   │                      ├─{gnome-terminal-}(3611)
│   │   │   │                      └─{gnome-terminal-}(5861)
```

```
│   │   │   │                      └─{pipewire-pulse}(2349)
│   │   │   ├─problem2(10149)
│   │   │   ├─snap(7219)─┬─{snap}(7226)
│   │   │   │           ├─{snap}(7227)
│   │   │   │           ├─{snap}(7228)
│   │   │   │           ├─{snap}(7229)
│   │   │   │           ├─{snap}(7230)
│   │   │   │           ├─{snap}(7231)
│   │   │   │           ├─{snap}(7232)
│   │   │   │           ├─{snap}(7233)
│   │   │   │           └─{snap}(8488)
│   │   │   ├─snapd-desktop-i(3200)───snapd-desktop-i(3272)─┬─{snapd-desktop-i}(3291)
│   │   │   │                                               ├─{snapd-desktop-i}(3292)
│   │   │   │                                               ├─{snapd-desktop-i}(3293)
```

Explanation:

In this program, a parent creates a child, and then the parent exits quickly. The child waits a bit using `sleep()`. Because the parent is gone, the child becomes an orphan process. That means the computer gives the child a new parent, usually a system process like `init`.

When the child prints its parent PID, it shows a different number (not the original parent). This proves that it became orphaned and was adopted by the system.

Answer - 3:

Problem:Write a C program to create a zombie process.

Code:

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

int main() {

    int pid;

    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) { // child process
        printf("Child process (PID: %d) created. Exiting...\n", getpid());
        // Child exits immediately, creating a zombie process
        exit(EXIT_SUCCESS);
    } else { // parent process
        printf("Parent process (PID: %d) created child with PID: %d\n",
getpid(), pid);
        // Parent sleeps to allow the child to become a zombie
        sleep(30);
        printf("Parent process exiting...\n");

    }



    exit(EXIT_SUCCESS);

}
```

[1], [5]

Output:

```
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$ gcc problem3.c -o problem3
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$ ./problem3
Parent process (PID: 10134) created child with PID: 10135
Child process (PID: 10135) created. Exiting...
Parent process exiting...
```

```
│   │   │   ├─gnome-shell-cal(2677)─┬─{gnome-shell-cal}(2678)
│   │   │   │                      ├─{gnome-shell-cal}(2679)
│   │   │   │                      ├─{gnome-shell-cal}(2681)
│   │   │   │                      ├─{gnome-shell-cal}(2682)
│   │   │   │                      ├─{gnome-shell-cal}(2683)
│   │   │   │                      └─{gnome-shell-cal}(2966)
│   │   │   ├─gnome-terminal-(3605)─┬─bash(3612)───pstree(10151)
│   │   │   │                      ├─bash(3729)───problem2(10148)───problem2(10149)
│   │   │   │                      ├─{gnome-terminal-}(3606)
│   │   │   │                      ├─{gnome-terminal-}(3607)
│   │   │   │                      ├─{gnome-terminal-}(3609)
│   │   │   │                      ├─{gnome-terminal-}(3610)
│   │   │   │                      ├─{gnome-terminal-}(3611)
│   │   │   │                      └─{gnome-terminal-}(5861)
```

Explanation:

In this program, the parent creates a child process using `fork()`. The child finishes its task quickly and exits, but the parent keeps running without using `wait()` to collect the child's exit status. That makes the child process become a zombie—it's finished, but still listed in the process table until the parent cleans it up. This shows how a zombie process exists briefly when the parent doesn't handle termination properly.

Answer - 4

Problem: Write a C program to create a main process named 'parent_process' having 3 child processes without any grandchildren processes. Child Processes' names are child_1, child_2, child_3. Trace the position in the process tree.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <sys/prctl.h>

int main() {
    prctl(PR_SET_NAME, "parent_process");  // Set parent process name
```

```
    printf("Parent process started. PID: %d\n", getpid());

    char *child_names[3] = {"child_1", "child_2", "child_3"};
    pid_t pids[3];

    for (int i = 0; i < 3; i++) {
        pids[i] = fork();

        if (pids[i] < 0) {
            perror("Fork failed");
            exit(EXIT_FAILURE);
        }

        if (pids[i] == 0) {
            prctl(PR_SET_NAME, child_names[i]);  // Set child process name
            printf("Hello from %s, PID: %d, PPID: %d\n", child_names[i],
getpid(), getppid());
            exit(EXIT_SUCCESS);
        }
    }

    sleep(20);
    // Parent waits for all children to finish
    for (int i = 0; i < 3; i++) {
        wait(NULL);
    }

    printf("Parent process: All child processes have finished.\n");

    exit(EXIT_SUCCESS);
}
```

[1], [5]

Output:

In this program, a main process named `parent_process` creates three child processes named `child_1`, `child_2`, and `child_3`. Each child is created using `fork()` and does a separate task. The parent and children all print their own Process ID (PID) and Parent Process ID (PPID), which helps trace their exact positions in the process tree.

Since each child is directly created by the main process and there are no grandchildren, the process tree stays flat. This makes it easy to confirm correct relationships like:

```
                                  └─{gnome-shell-cal}(2636)
                ├─gnome-terminal-(6049)─┬─bash(6070)───parent_process(12247)─┬─child_1(12248)
                                        │                                    ├─child_2(12249)
                                        │                                    └─child_3(12250)
                                        ├─bash(10070)───pstree(12252)
                                        ├─{gnome-terminal-}(6060)
                                        ├─{gnome-terminal-}(6061)
                                        ├─{gnome-terminal-}(6063)
                                        ├─{gnome-terminal-}(6064)
                                        ├─{gnome-terminal-}(6069)
                                        └─{gnome-terminal-}(10069)
```

Answer - 5

Problem: Write a C program to create a main process named 'parent_process' having 'n' child processes without any grandchildren processes. Child Processes' names are child_1, child_2, child_3,....., child_n. Trace the position in the process tree. Number of child processes (n) and name of child processes will be given in the CLI of Linux based systems. Example: $ ./parent_process 3 child_1 child_2 child_3

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/prctl.h>

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <n> <child_1> <child_2> ... <child_n>\n",
argv[0]);
```

```
        exit(EXIT_FAILURE);
    }

    int n = atoi(argv[1]);

    if (argc != n + 2) {
        printf("Error: You must enter %d child names.\n", n);
        exit(EXIT_FAILURE);
    }

    prctl(PR_SET_NAME, "parent_process");  // Rename main process

    printf("Parent (PID: %d) started.\n", getpid());

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            exit(EXIT_FAILURE);
        }

        if (pid == 0) {
            // In child process
            prctl(PR_SET_NAME, argv[i + 2]);  // Set child name
            printf("Child %d — Name: %s, PID: %d, PPID: %d\n",
                   i + 1, argv[i + 2], getpid(), getppid());
            exit(EXIT_SUCCESS);  // No grandchildren
        }
    }

    sleep(20);
    // Parent waits for all children
    for (int i = 0; i < n; i++) {
        wait(NULL);
    }

    printf("Parent done. All %d children finished.\n", n);
    exit(EXIT_SUCCESS);
}
```

[1], [5]

Output:

In this program, the main process called `parent_process` creates ‘n’ child processes using a loop and `fork()`. The number of child processes (n) and their names (like `child_1`, `child_2`, ...) are passed through the command line when running the program in Linux. Each child prints its own PID, its parent’s PID, and its given name, showing they are all directly created by the parent. There are no grandchildren, and the process tree stays flat.

```
│   │   │   ├─gnome-terminal-(6049)─┬─bash(6070)───parent_process(13110)─┬─child_1(13111)
│   │   │   │                      │                                    ├─child_2(13112)
│   │   │   │                      │                                    └─child_3(13113)
│   │   │   │                      ├─bash(10070)───pstree(13114)
│   │   │   │                      ├─{gnome-terminal-}(6060)
│   │   │   │                      ├─{gnome-terminal-}(6061)
│   │   │   │                      ├─{gnome-terminal-}(6063)
│   │   │   │                      ├─{gnome-terminal-}(6064)
│   │   │   │                      ├─{gnome-terminal-}(6069)
│   │   │   │                      └─{gnome-terminal-}(10069)
```

Answer - 7

Problem: Write a C program to analyze the effect of local and global variables on a parent process and a child process.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int global_var = 10; // Global variable

int main() {
    int local_var = 20; // Local variable

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
```

```
    if (pid == 0) {
        // Child process
        global_var += 5;
        local_var += 5;
        printf("Child process:\n");
        printf("  global_var = %d\n", global_var);
        printf("  local_var  = %d\n", local_var);
    } else {
        // Parent process waits for child to finish
        wait(NULL);
        printf("Parent process:\n");
        printf("  global_var = %d\n", global_var);
        printf("  local_var  = %d\n", local_var);
    }

    exit(EXIT_SUCCESS);
}
```

[1], [2]

Output:

```
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$ ./problem7
Child process:
  global_var = 15
  local_var  = 25
Parent process:
  global_var = 10
  local_var  = 20
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$
```

Explanation:

This program shows how local and global variables work in parent and child processes. After using `fork()`, both the parent and child get their own copies of the variables. If one changes a value, it does not affect the other. This proves that variables are not shared—they're copied separately when a new process is created.

Answer - 8

Problem: Write a C program to show how two related processes can communicate with each other by an unnamed pipe.

Code:

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>
#include <sched.h>
#include <string.h>

void main(){
  pid_t pid;
  int processor_no;
  int pipe_fd[2];// Pipe Descriptor
  char buffer[80];


  pipe(pipe_fd);

  //Create a child
  pid = fork();

  if(pid < 0){
    perror("Fork");
    exit(EXIT_FAILURE);
  }else if(pid == 0){ //Child process
    pid = getpid();

    char *msg = "Type a message for the Parent: ";
    write(1, msg, strlen(msg));
    read(0, buffer, sizeof(buffer));

    //Write Child message to the pipe
    close(pipe_fd[0]);
    write(pipe_fd[1], buffer, strlen(buffer));
```

```
    close(pipe_fd[1]);

  }else{  //Parent Process

    //Read Child's message from the pipe
    close(pipe_fd[1]);
    read(pipe_fd[0], buffer, sizeof(buffer));
    close(pipe_fd[0]);

    char *msg = "Parent received message: ";
    write(1, msg, strlen(msg));
    write(1, buffer, strlen(buffer));
    int status;
    wait(&status);
  }
  exit(EXIT_SUCCESS);
}
```

[1],[2]

Output:

In this program, two related processes—a parent and its child—talk to each other using an unnamed pipe. The parent sends a message through the pipe, and the child reads it. Pipes allow them to share data without using files. Since they are related (created using fork()), they can use this pipe directly. This shows how a pipe helps two processes communicate while the program is running.

```
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$ ./problem8
Type a message for the Parent: hello there
Received message: hello there
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$
```

Answer - 9

Problem: Write a C program to show how two unrelated processes can communicate with each other by a named pipe.

Code:

Reader.c:

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main() {
    char buffer[80];

    // Open named pipe in read mode
    char *named_pipe_path = "/home/mohon/temp/named_pipe"; // Path to the
named pipe
    int fd;
    fd = open(named_pipe_path, O_RDONLY); // Open named pipe in read mode
    if (fd < 0) {
        perror("Error opening named pipe");
        exit(EXIT_FAILURE);
    }


    // Print the received message
    char *msg = "Received message: ";

    read(fd , buffer, sizeof(buffer)); // Read from the named pipe
    write(1, msg, strlen(msg));
    write(1, buffer, strlen(buffer));

    close(fd); // Close the file descriptor

    exit(EXIT_SUCCESS); // Exit the process
}
```

[2], [5], [3]

Writer.c:

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main(){
    char buffer[80];
    char *msg = "Type a message for sending reader: ";

    write(1, msg, strlen(msg)); // Write to stdout
    read(0, buffer, sizeof(buffer)); // Read from stdin


    //Open nammed pipe in read mode
    char *named_pipe_path = "/home/mohon/temp/named_pipe"; // Path to the
named pipe
    int fd;
    fd = open(named_pipe_path, O_WRONLY); // Open named pipe in write mode
    if( fd < 0){
        perror("Error opening named pipe");
        exit(EXIT_FAILURE);
    }
    write(fd, buffer, strlen(buffer)); // Write to the named pipe
    close(fd); // Close the file descriptor



    exit(EXIT_SUCCESS); // Exit the process
}
```

[2], [5], [3]

Output:

This program shows how two unrelated processes—not created using `fork()`—can share data using a named pipe (also called FIFO). A named pipe is a file in the system that both processes can open. One process writes a message to it, and the other process reads from it. Since they aren't parent-child, this method lets them communicate even though they're separate.

This proves how named pipes allow indirect communication between unrelated programs.

Writer screen:

This program shows how **two unrelated processes**—not created using `fork()`—can share data using a **named pipe** (also called FIFO). A named pipe is a file in the system that both processes can open. One process **writes** a message to it, and the other process **reads** from it. Since they aren't parent-child, this method lets them **communicate** even though they're separate.

This proves how named pipes allow **indirect communication** between unrelated programs.

```
mohon@abc123:~/temp$ ./writer
Type a message for sending reader: Hello, I am writter
mohon@abc123:~/temp$ ./writer
Type a message for sending reader: Which cpu you use?
mohon@abc123:~/temp$
```

Reader screen:

```
mohon@abc123:~/temp$ ./reader
Received message: Hello, I am writter
mohon@abc123:~/temp$ ./reader
Received message: Which cpu you use?
mohon@abc123:~/temp$
```

[2], [5], [3]

Answer - 10

Problem: Write a C program to show how two related processes can communicate with each other by a named pipe.

Code:

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <sys/stat.h>

int main() {
    char *named_pipe_path = "/home/mohon/temp/named_pipe";
    char buffer[80];
    int fd;

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }

    else if (pid == 0) {
        // Child Process: Reader
        sleep(1); // Ensure writer has opened the pipe

        fd = open(named_pipe_path, O_RDONLY); // Open pipe in read mode
        if (fd < 0) {
            perror("Error opening named pipe");
            exit(EXIT_FAILURE);
        }
```

```
        read(fd, buffer, sizeof(buffer)); // Read message from pipe
        char *msg = "Child received: ";
        write(1, msg, strlen(msg));
        write(1, buffer, strlen(buffer));

        close(fd);
        exit(EXIT_SUCCESS);
    }

    else {
        // Parent Process: Writer
        char *prompt = "Parent: Type a message for child: ";
        write(1, prompt, strlen(prompt));
        read(0, buffer, sizeof(buffer)); // Read from stdin

        fd = open(named_pipe_path, O_WRONLY); // Open pipe in write mode
        if (fd < 0) {
            perror("Error opening named pipe");
            exit(EXIT_FAILURE);
        }

        write(fd, buffer, strlen(buffer)); // Write message to pipe
        close(fd);
        wait(NULL); // Wait for child to finish
    }



    return 0;
}
```

[2], [5], [3]

Output:

```
mohon@abc123:~/temp$ gcc problem10.c -o problem10
mohon@abc123:~/temp$ ./problem10
Parent: Type a message for child: Hello, I am parent process
Child received: Hello, I am parent process
mohon@abc123:~/temp$
```

Explanation:

This program shows how a parent and its child process can talk to each other using a named pipe (also called FIFO). A named pipe is a special file created using the `mkfifo()` function. After `fork()`:

- One process writes a message to the pipe.
- The other process reads it from the pipe.

Since the processes are related, they can easily use this named pipe to share information while running. This proves that named pipes work for direct communication between parent and child processes.

Answer - 11

Problem: Write a C program to show how two unrelated processes can communicate with each other by a message queue.

Code:

Message_sender.c:

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>
#include <fcntl.h>

struct msg_queue {
    long type;
    char text[80];
};

int main() {
    key_t key = ftok("test.c", 65);

    if (key == -1) {
        perror("ftok failed");
        exit(EXIT_FAILURE);
    }
```

```
    int msqid = msgget(key, IPC_CREAT | 0666);
    if (msqid == -1) {
        perror("msgget failed");
        exit(EXIT_FAILURE);
    }

    struct msg_queue msg;
    msg.type = 1;

    char *prompt = "Enter a message to send: ";
    write(1, prompt, strlen(prompt));  // Write prompt to stdout

    // Read user input from stdin (file descriptor 0)
    read(0, msg.text, sizeof(msg.text));


    // Send message to queue
    if (msgsnd(msqid, &msg, strlen(msg.text), 0) == -1) {
        perror("msgsnd failed");
        exit(EXIT_FAILURE);
    }

    write(1, "Message sent successfully.\n", 28);

    return 0;
}
```

[2], [5], [3]

Message_receiver.c:

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>

struct msg_queue {
    long type;
    char text[80];
};
```

```
int main() {
    key_t key = ftok("test.c", 65); // Must match the sender's key

    if (key == -1) {
        perror("ftok failed");
        exit(EXIT_FAILURE);
    }

    int msqid = msgget(key, 0666);
    if (msqid == -1) {
        perror("msgget failed");
        exit(EXIT_FAILURE);
    }

    struct msg_queue msg;

    if (msgrcv(msqid, &msg, sizeof(msg.text), 1, 0) == -1) {
        perror("msgrcv failed");
        exit(EXIT_FAILURE);
    }

    printf("Received message: %s", msg.text);

    // Optionally remove the queue
    if (msgctl(msqid, IPC_RMID, NULL) == -1) {
        perror("msgctl (remove queue) failed");
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

[2], [5], [3]

Output:

This program shows how two separate processes—not created using `fork()`—can communicate using a message queue. Message queues allow one process to send data, and another process to receive it, even if they're not related. The queue acts like a mailbox: one process drops a message in, and the other picks it up. This proves that message queues are useful for sharing data between unrelated programs.

```
mohon@abc123:~/temp$ gcc problem11_message_sender.c -o problem11_message_sender
mohon@abc123:~/temp$ ./problem11_message_sender
Enter a message to send: Hello, I am from unrelated process
Message sent successfully.
mohon@abc123:~/temp$
```

Answer - 12

Problem: Write a C program to show how two related processes can communicate with each other by a message queue.

Code:

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>

int main() {
    // Create a message queue
    key_t key = ftok("message_queue.c", 24);
    //printf("Key : %d\n", key);

    int msqid = msgget(key, IPC_CREAT | 0666);
    if (msqid == -1) {
        perror("msgget failed");
        exit(EXIT_FAILURE);
    }

    struct msg_queue {
        long type;
        char text[80];
    } msg;



    pid_t pid = fork();
```

```
if (pid == -1) {  // Fork failed
    perror("fork failed");
    exit(EXIT_FAILURE);
}

if (pid == 0) { // Child process
    msg.type = 1;  // Child sends message of type 1

    char *prompt = "Child: Enter a message to send to parent: ";
    write(1, prompt, strlen(prompt));  // Write prompt to stdout
    read(0, msg.text, sizeof(msg.text));

    // Send the message
    if (msgsnd(msqid, &msg, sizeof(msg), 0) == -1) {
        perror("msgsnd failed");
        exit(EXIT_FAILURE);
    }

} else { // Parent process
    int status;
    wait(&status);  // Wait for the child to finish


    // Receive the message
    if (msgrcv(msqid, &msg, sizeof(msg), 1, 0) == -1) {
        perror("msgrcv failed");
        exit(EXIT_FAILURE);
    }

    printf("Parent received: %s\n", msg.text);

    // Remove the message queue
    if (msgctl(msqid, IPC_RMID, 0) == -1) {
        perror("msgctl failed");
        exit(EXIT_FAILURE);
    }
}

exit(EXIT_SUCCESS);
```

```
}
```

[2], [5], [3]

Explanation:

In this program, a parent process uses `fork()` to create a child process, and they talk to each other using a message queue. A message queue is a system feature that lets one process send a message, and the other receive it—kind of like sending a letter to your teammate. Since both processes are related, this method helps them share data safely and directly. This proves message queues work well for communication between parent and child.

Output screen:

```
mohon@abc123:~/temp$ gcc problem12_message_queue.c -o p12
mohon@abc123:~/temp$ ./p12
Child: Enter a message to send to parent: Hello i am from child
Parent received: Hello i am from child
```

Answer - 13

Problem:Write a C program to show how data inconsistency arises in a multi-threaded process.

Code:

```
#include <stdio.h>
#include <pthread.h>

int count = 0; // Shared variable

void* increment1(void* arg) {
    for (int i = 0; i < 10; i++) {
        count++; // No synchronization
        printf("Thread 1: count = %d\n", count);
    }
    return NULL;
}

void* increment2(void* arg) {
    for (int i = 0; i < 10; i++) {
        count--; // No synchronization
        printf("Thread 2: count = %d\n", count);
    }
    return NULL;
```

```
}



int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment1, NULL);
    pthread_create(&t2, NULL, increment2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final count: %d\n", count);

    return 0;
}
```

[2], [5], [4]

Explanation:

This program shows how multiple threads in one process can cause data inconsistency. If two or more threads try to change the same variable at the same time, they can mess up the result. This happens when there's no proper control (like using mutex or locks). For example, if two threads both try to increase a counter, the final value might be wrong because they both read and write without waiting. This proves that multi-threaded programs must use safety techniques to avoid unexpected results.

Output:

```
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$ gcc problem13.c -o problem13
mohon@mohon-Inspiron-15-3511:~/Documents/courses/part_iii_even/os_lab$ ./problem13
Thread 1: count = 1
Thread 1: count = 1
Thread 1: count = 2
Thread 2: count = 0
Thread 2: count = 2
Thread 2: count = 1
Thread 2: count = 0
Thread 2: count = -1
Thread 2: count = -2
Thread 2: count = -3
Thread 2: count = -4
Thread 2: count = -5
Thread 2: count = -6
Thread 1: count = 3
Thread 1: count = -5
Thread 1: count = -4
Thread 1: count = -3
Thread 1: count = -2
Thread 1: count = -1
Thread 1: count = 0
Final count: 0
```

Answer - 14

Problem: Write a C program to show how data inconsistency arises in two related processes (e.g., parent & child processes) when they share a memory space

Code:

```c
#define _GNU_SOURCE
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <sys/shm.h>

int main(){
    // Shared memory pointer and loop variable
    int *shmaddr;
    int i;

    // Create a shared memory segment
    int shmid;
    key_t key = 5678;
```

```
    shmid = shmget(key, sizeof(int), IPC_CREAT | 0666);
    if(shmid == -1){
        perror("shmget: ");
        exit(EXIT_FAILURE);
    }

    // Create a child process
    pid_t pid;
    pid = fork();
    if(pid == -1){
        perror("fork: ");
        exit(EXIT_FAILURE);
    }

    if(pid == 0){ // Child process
        shmaddr = shmat(shmid, NULL, 0);
        if(shmaddr == (int*) -1){
            perror("shmat (child): ");
            exit(EXIT_FAILURE);
        }

        for(i = 0; i < 10; i++){
            printf("Child before write: shared_var = %d\n", *shmaddr);
            *shmaddr = 100;
            printf("Child after  write: shared_var = %d\n", *shmaddr);
        }

        if(shmdt(shmaddr) == -1){
            perror("shmdt (child): ");
            exit(EXIT_FAILURE);
        }
    }
    else{ // Parent process
        shmaddr = shmat(shmid, NULL, 0);
        if(shmaddr == (int*) -1){
            perror("shmat (parent): ");
            exit(EXIT_FAILURE);
        }

        for(i = 0; i < 10; i++){
```

```
            printf("Parent before write: shared_var = %d\n", *shmaddr);
            *shmaddr = 999;
            printf("Parent after  write: shared_var = %d\n", *shmaddr);
        }

        int status;
        wait(&status);

        printf("Parent final read   : shared_var = %d\n", *shmaddr);

        if(shmdt(shmaddr) == -1){
            perror("shmdt (parent): ");
            exit(EXIT_FAILURE);
        }

        if(shmctl(shmid, IPC_RMID, NULL) == -1){
            perror("shmctl: ");
            exit(EXIT_FAILURE);
        }
    }

    exit(EXIT_SUCCESS);
}
```

[2], [5], [3]

Explanation:

This program shows how a **parent and child process** can share data using **System V shared memory** (`shmget()`, `shmat()`, etc.). After using `fork()`, both processes access the **same memory space**. If they both try to **change the shared variable at the same time** without any control (like using semaphores), the final result can be wrong—this is called **data inconsistency**. It happens because both processes can **overwrite each other's changes**, leading to incorrect values.

Output:

```
mohon@abc123:~/temp$ gcc problem14.c -o problem14
mohon@abc123:~/temp$ ./problem14
Parent before write: shared_var = 0
Child before write: shared_var = 999
Child after  write: shared_var = 100
Child before write: shared_var = 100
Parent after  write: shared_var = 100
Parent before write: shared_var = 100
Child after  write: shared_var = 999
Child before write: shared_var = 999
Parent after  write: shared_var = 100
Parent before write: shared_var = 100
Child after  write: shared_var = 999
Child before write: shared_var = 999
Parent after  write: shared_var = 100
Parent before write: shared_var = 100
Child after  write: shared_var = 999
Child before write: shared_var = 999
Parent after  write: shared_var = 100
Parent before write: shared_var = 100
Child after  write: shared_var = 999
Child before write: shared_var = 999
Parent after  write: shared_var = 100
Parent before write: shared_var = 100
Child after  write: shared_var = 999
Child before write: shared_var = 999
Child after  write: shared_var = 100
Child before write: shared_var = 100
```

Answer - 15

Problem: Write a C program to show how data inconsistency arises in two unrelated processes when they share a memory space.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>

#define SHM_KEY 1234

int main() {
```

```
    int *shmaddr;
    int shmid;
    int i;

    // Create or get shared memory
    shmid = shmget(SHM_KEY, sizeof(int), IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget");
        exit(1);
    }

    // Attach to shared memory
    shmaddr = shmat(shmid, NULL, 0);
    if (shmaddr == (int *) -1) {
        perror("shmat");
        exit(1);
    }

    // Write to shared memory
    for (i = 0; i < 100; i++) {
        printf("Before write: shared_var = %d\n", *shmaddr);
        (*shmaddr )++;  // Write random value
        usleep(100000);            // Small delay to allow overlap
        printf("After  write: shared_var = %d\n", *shmaddr);
    }

    // Detach from shared memory
    if (shmdt(shmaddr) == -1) {
        perror("shmdt");
        exit(1);
    }

    if (shmctl(shmid, IPC_RMID, NULL) == 0) {
        printf("Shared memory segment deleted.\n");
    }

    exit(EXIT_SUCCESS);
}
```

[1], [5], [3]

Explanation:

This code shows how two separate programs (not parent-child) can share data using System V shared memory. The program writes to a shared integer 100 times with small delays (`usleep`). If another unrelated program also changes the same shared variable at the same time, the result may be wrong. For example, two processes may both read 5, increase it, and both write 6—missing one update. This is called data inconsistency, and it happens because no protection (like semaphores) is used to control access.

Output:

```
mohon@abc123:~/temp$ gcc problem15.c -o problem15
mohon@abc123:~/temp$ ./problem15
Before write: shared_var = 9531
After  write: shared_var = 9532
Before write: shared_var = 9532
After  write: shared_var = 9533
Before write: shared_var = 9533
After  write: shared_var = 9534
Before write: shared_var = 9534
After  write: shared_var = 9535
Before write: shared_var = 9535
After  write: shared_var = 9536
Before write: shared_var = 9536
After  write: shared_var = 9537
Before write: shared_var = 9537
After  write: shared_var = 9538
Before write: shared_var = 9538
After  write: shared_var = 9539
Before write: shared_var = 9539
After  write: shared_var = 9540
Before write: shared_var = 9540
After  write: shared_var = 9541
Before write: shared_var = 9541
After  write: shared_var = 9542
Before write: shared_var = 9542
After  write: shared_var = 9543
Before write: shared_var = 9543
```

```
mohon@abc123:~/temp$ ./problem15
Before write: shared_var = 9558
After  write: shared_var = 9560
Before write: shared_var = 9560
After  write: shared_var = 9562
Before write: shared_var = 9562
After  write: shared_var = 9564
Before write: shared_var = 9564
After  write: shared_var = 9566
Before write: shared_var = 9566
After  write: shared_var = 9568
Before write: shared_var = 9568
After  write: shared_var = 9570
Before write: shared_var = 9570
After  write: shared_var = 9572
Before write: shared_var = 9572
After  write: shared_var = 9574
Before write: shared_var = 9574
After  write: shared_var = 9576
Before write: shared_var = 9576
After  write: shared_var = 9578
Before write: shared_var = 9578
After  write: shared_var = 9580
Before write: shared_var = 9580
After  write: shared_var = 9582
Before write: shared_var = 9582
After  write: shared_var = 9584
```

Answer - 16

Problem: Write a C program to handle racing situations in a multi-threaded process.

Code:

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>

int x = 0;
pthread_mutex_t key;


void *addition(){
```

```
        int processor_no;

        for(int i=0; i<100 ; i++){
        processor_no = sched_getcpu();
            pthread_mutex_lock(&key);
        x++;
        printf("Addition thread, Processor no: %d, x: %d\n", processor_no,
x);
            pthread_mutex_unlock(&key);
        }
}

void *subtraction(){
        int processor_no;

        for(int i=0; i<100 ; i++){
        processor_no = sched_getcpu();
            pthread_mutex_lock(&key);
        x--;
        printf("Subtraction thread, Processor no: %d, x: %d\n",
processor_no, x);
            pthread_mutex_unlock(&key);
        }


}




void main(){
        pthread_t  tid1, tid2;
        pthread_create(&tid1, NULL, &addition, NULL);
        pthread_create(&tid2, NULL, &subtraction, NULL);
        int processor_no;

        for(int i=0; i<100 ; i++){
        processor_no = sched_getcpu();
        printf("Main thread, Processor no: %d\n", processor_no);
        }
```

```
        // To force to main thread to wait until sub-thread is complete
        pthread_join(tid1, NULL);
        pthread_join(tid2, NULL);

}
```

[2], [5], [3]

Explanation:

In a multi-threaded program, when two or more threads try to change the same variable at the same time, it can lead to a racing problem. That means the output can be wrong or unpredictable because the threads are fighting to update the value first. This program solves that problem using a mutex (mutual exclusion lock). The mutex makes sure only one thread can access the variable at a time, which prevents inconsistency and keeps the data safe.

Output:

```
mohon@abc123:~/temp$ gcc problem16.c -o problem16
mohon@abc123:~/temp$ ./problem16
Addition thread, Processor no: 3, x: 1
Addition thread, Processor no: 3, x: 2
Addition thread, Processor no: 3, x: 3
Addition thread, Processor no: 3, x: 4
Addition thread, Processor no: 3, x: 5
Addition thread, Processor no: 3, x: 6
Addition thread, Processor no: 3, x: 7
Addition thread, Processor no: 3, x: 8
Addition thread, Processor no: 3, x: 9
Addition thread, Processor no: 3, x: 10
Subtraction thread, Processor no: 5, x: 9
Subtraction thread, Processor no: 5, x: 8
Subtraction thread, Processor no: 5, x: 7
Addition thread, Processor no: 3, x: 8
Main thread, Processor no: 1
Main thread, Processor no: 1
Addition thread, Processor no: 3, x: 9
Main thread, Processor no: 1
Main thread, Processor no: 1
Main thread, Processor no: 1
Main thread, Processor no: 1
Main thread, Processor no: 1
Main thread, Processor no: 1
Main thread, Processor no: 1
```

Answer - 17

Problem: Write a C program to handle racing situations in multiple unrelated processes.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>

#define SHM_KEY 1234
#define SEM_NAME "/my_named_sem"

int main() {
    // Create or get shared memory
    int shmid = shmget(SHM_KEY, sizeof(int), IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget");
        exit(1);
    }

    // Attach to shared memory
    int *shared_val = (int *)shmat(shmid, NULL, 0);
    if (shared_val == (void *)-1) {
        perror("shmat");
        exit(1);
    }

    // Open or create named semaphore
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
    if (sem == SEM_FAILED) {
        perror("sem_open");
        exit(1);
    }

    // Do work with synchronization
    for (int i = 0; i < 20; i++) {
        sem_wait(sem); // Lock
```

```
        (*shared_val)++;
        printf("PID %d updated value to %d\n", getpid(), *shared_val);
        sem_post(sem); // Unlock
        usleep(100000); // 100ms
    }

    // Detach shared memory
    shmdt(shared_val);

    // Close semaphore
    sem_close(sem);

    return 0;
}
```

[2], [5], [4]

Explanation:

This program shows how two separate (unrelated) processes share data using System V shared memory (`shmget()`, `shmat()`), and how they can face racing problems. If both processes try to change the same shared variable at the same time, the result can be wrong or inconsistent. To fix this, the program uses a System V semaphore (`semget()`, `semop()`) to control access. The semaphore works like a lock—only one process at a time can update the shared data, keeping it safe and correct.

Output:

```
mohon@abc123:~/temp$ ./p17
PID 2100 updated value to 61
PID 2100 updated value to 62
PID 2100 updated value to 63
PID 2100 updated value to 64
PID 2100 updated value to 65
PID 2100 updated value to 66
PID 2100 updated value to 68
PID 2100 updated value to 70
PID 2100 updated value to 72
PID 2100 updated value to 74
PID 2100 updated value to 76
PID 2100 updated value to 78
PID 2100 updated value to 80
PID 2100 updated value to 82
PID 2100 updated value to 84
PID 2100 updated value to 86
PID 2100 updated value to 88
PID 2100 updated value to 90
PID 2100 updated value to 92
PID 2100 updated value to 94
```

```
mohon@abc123:~/temp$ ./p17
PID 2101 updated value to 67
PID 2101 updated value to 69
PID 2101 updated value to 71
PID 2101 updated value to 73
PID 2101 updated value to 75
PID 2101 updated value to 77
PID 2101 updated value to 79
PID 2101 updated value to 81
PID 2101 updated value to 83
PID 2101 updated value to 85
PID 2101 updated value to 87
PID 2101 updated value to 89
PID 2101 updated value to 91
PID 2101 updated value to 93
PID 2101 updated value to 95
PID 2101 updated value to 96
PID 2101 updated value to 97
PID 2101 updated value to 98
PID 2101 updated value to 99
PID 2101 updated value to 100
```

Answer - 18

Problem: Write a C program for creating a multi-threaded process and check: A. If one thread in the process calls fork(), does the new process duplicate all threads, or is the new process single-threaded? B. If a thread invokes the exec() system call, does it replace the entire code of the process? C. If exec() is called immediately after forking, will all threads be duplicated?

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <fcntl.h>

#define SEM_NAME "/counter_sem"

int main() {
    // Create a counting semaphore with initial value 2 (2 resources)
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0666, 2);
    if (sem == SEM_FAILED) {
        perror("sem_open");
        exit(1);
    }

    printf("Process %d waiting for resource...\n", getpid());
    sem_wait(sem); // Try to acquire one unit

    printf("Process %d got resource!\n", getpid());
    sleep(2); // Simulate doing some work

    printf("Process %d releasing resource.\n", getpid());
    sem_post(sem); // Release the unit

    sem_close(sem); // Close the semaphore (doesn't delete it)a

    return 0;
}
```

[1], [5], [3]

Explanation:

This program uses a System V counter semaphore to control access to a shared resource. A counter semaphore can be set to a number greater than 1, which means it allows multiple processes to access the resource at the same time—but only up to a safe limit. Each process decreases the counter when entering and increases it when done. If the counter reaches 0, other processes have to wait until someone releases the resource. This proves that semaphores help manage resource sharing and prevent conflicts when several processes work together.

Output:

```
mohon@abc123:~/temp$ ./p18
Process 2187 waiting for resource...
Process 2187 got resource!
Process 2187 releasing resource.
mohon@abc123:~/temp$ ./p18
Process 2188 waiting for resource...
Process 2188 got resource!
        Process 2188 releasing resource.
mohon@abc123:~/temp$ ./p18
Process 2191 waiting for resource...
Process 2191 got resource!
```

```
mohon@abc123:~/temp$ ./p18
Process 2186 waiting for resource...
Process 2186 got resource!
Process 2186 releasing resource.
mohon@abc123:~/temp$ ./p18
Process 2189 waiting for resource...
Process 2189 got resource!
Process 2189 releasing resource.
mohon@abc123:~/temp$ ./p18
Process 2192 waiting for resource...
Process 2192 got resource!
Process 2192 releasing resource.
```

```
mohon@abc123:~/temp$ ./p18
Process 2185 waiting for resource...
Process 2185 got resource!
Process 2185 releasing resource.
mohon@abc123:~/temp$ ./p18
Process 2190 waiting for resource...
Process 2190 got resource!
Process 2190 releasing resource.
mohon@abc123:~/temp$ ./p18
Process 2193 waiting for resource...
Process 2193 got resource!
Process 2193 releasing resource.
```

Answer - 19

Problem: This program uses a System V counter semaphore to control access to a shared resource. A counter semaphore can be set to a number greater than 1, which means it allows multiple processes to access the resource at the same time—but only up to a safe limit. Each process decreases the counter when entering and increases it when done. If the counter reaches 0, other processes have to wait until someone releases the resource. This proves that semaphores help manage resource sharing and prevent conflicts when several processes work together.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/wait.h>

void* threadA_func(void* arg) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child from fork(): only one thread exists here\n");
        exit(0);
    } else {
        printf("Parent after fork()\n");
    }
```

```
}
void* threadB_func(void* arg) {
    printf("Thread calling exec()\n");
    execl("./hello_program", "hello_program", NULL); // Custom Hello World
program
    perror("exec failed");

}

void* threadC_func(void* arg) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child after fork(), now calling exec()\n");
        execl("./hello_program", "hello_program", NULL);
        perror("exec failed");
        exit(1);
    } else {
        printf("Parent after fork from thread C\n");
    }

}

int main() {
    pthread_t tA, tB, tC;

    pthread_create(&tA, NULL, threadA_func, NULL);
    pthread_join(tA, NULL); // Case A



    pthread_create(&tC, NULL, threadC_func, NULL);
    pthread_join(tC, NULL); // Case C

    pthread_create(&tB, NULL, threadB_func, NULL);
    pthread_join(tB, NULL); // Case B — may not return if exec succeeds


}
```

[1], [5], [3]

Explanation:

- **A. `fork()` called by a thread : Only that thread is copied; child becomes single-threaded.**
- **B. `exec()` called by any thread : Replaces the entire program; all threads are lost.**
- **C. `exec()` after `fork()` : Child is single-threaded before `exec()`, so final process stays single-threaded.**

Output:

```
mohon@abc123:~/temp$ ./p19
Parent after fork()
Child from fork(): only one thread exists here
Child after fork(), now calling exec()
Parent after fork from thread C
Thread calling exec()
Hello, Mohon! This is your new exec program.
Hello, Mohon! This is your new exec program.
mohon@abc123:~/temp$
```

Answer - 20

Problem: Write a multithreaded program that calculates various statistical values for a list of numbers. This program will be passed a series of numbers on the command line and will then create three separate worker threads. One thread will determine the average of the numbers, the second will determine the maximum value, and the third will determine the minimum value. For example, suppose your program is passed the integers 90 81 78 95 79 72 85 The program will report A. The average value is 82 B. The minimum value is 72 C. The maximum value is 95 The variables representing the average, minimum, and maximum values will be stored globally. The worker threads will set these values, and the parent thread will output the values once the workers have exited.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int average;
int minimum;
int maximum;

int *numbers;
```

```
int count;

// Function to calculate average
void* calc_average(void* arg) {
    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += numbers[i];
    }
    average = sum / count;
    pthread_exit(NULL);
}

// Function to find minimum
void* calc_minimum(void* arg) {
    minimum = numbers[0];
    for (int i = 1; i < count; i++) {
        if (numbers[i] < minimum)
            minimum = numbers[i];
    }
    pthread_exit(NULL);
}

// Function to find maximum
void* calc_maximum(void* arg) {
    maximum = numbers[0];
    for (int i = 1; i < count; i++) {
        if (numbers[i] > maximum)
            maximum = numbers[i];
    }
    pthread_exit(NULL);
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        printf("Usage: %s num1 num2 ...\n", argv[0]);
        return 1;
    }

    count = argc - 1;
    numbers = malloc(count * sizeof(int));
```

```
    for (int i = 0; i < count; i++) {
        numbers[i] = atoi(argv[i + 1]);
    }

    pthread_t tid1, tid2, tid3;

    // Create threads
    pthread_create(&tid1, NULL, calc_average, NULL);
    pthread_create(&tid2, NULL, calc_minimum, NULL);
    pthread_create(&tid3, NULL, calc_maximum, NULL);

    // Wait for threads to complete
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    pthread_join(tid3, NULL);

    // Output results
    printf("The average value is %d\n", average);
    printf("The minimum value is %d\n", minimum);
    printf("The maximum value is %d\n", maximum);

    free(numbers);
    return 0;
}
```

[1], [5], [3]

Explanation:

This program takes numbers from the command line and creates three threads:

- One thread finds the average
- One thread finds the maximum
- One thread finds the minimum

All results are stored in global variables. The main thread waits for the three threads to finish, then prints the results. This shows how multi-threading can be used to do parallel calculations and share the final output safely.

Output:

```
mohon@abc123:~/temp$ gcc p20.c -o p20
mohon@abc123:~/temp$ ./p20 90 81 78 95 79 72 85
The average value is 82
The minimum value is 72
The maximum value is 95
mohon@abc123:~/temp$ ./p20 1 2 3 4 5 6
The average value is 3
The minimum value is 1
The maximum value is 6
mohon@abc123:~/temp$ ./p20 10 15 20 50 100
The average value is 39
The minimum value is 10
The maximum value is 100
mohon@abc123:~/temp$
```

Answer - 21

Problem: The Fibonacci sequence is the series of numbers 0, 1, 1, 2, 3, 5, 8, ... Formally, it can be expressed as: fib0 = 0 fib1 = 1 fibn = fibn−1 + fibn−2 Write a multithreaded program that generates the Fibonacci sequence. This program should work as follows: On the command line, the user will enter the number of Fibonacci numbers that the program is to generate. The program will then create a separate thread that will generate the Fibonacci numbers, placing the sequence in data that can be shared by the threads (an array is probably the most convenient data structure). When the thread finishes execution, the parent thread will output the sequence generated by the child thread. Because the parent thread cannot begin outputting the Fibonacci sequence until the child thread finishes, the parent thread will have to wait for the child thread to finish.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int fib[100];
int n;
```

```
void* generate_fib(void* arg) {
    fib[0] = 0;
    if (n > 1)
        fib[1] = 1;

    for (int i = 2; i < n; i++)
        fib[i] = fib[i - 1] + fib[i - 2];

    pthread_exit(0);
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        printf("Usage: ./fib_thread <number_of_terms>\n");
        return 1;
    }

    n = atoi(argv[1]);
    if (n < 0) {
        printf("Please enter a positive number.\n");
        return 1;
    }

    pthread_t t1;

    pthread_create(&t1, NULL, generate_fib, NULL);
    pthread_join(t1, NULL);  // Wait for child thread to finish

    printf("Fibonacci sequence up to %d terms:\n", n);
    for (int i = 0; i < n; i++)
        printf("%d ", fib[i]);

    printf("\n");
    return 0;
}
```

Explanation:

This program creates a thread to generate Fibonacci numbers, based on a number given in the command line. The thread stores the sequence in a shared array, and the main thread waits until it finishes. After that, the main thread prints the full sequence. This shows how threads can work in parallel, and how one thread can wait for another to finish before using its output.

Output:

```
mohon@abc123:~/temp$ gcc p21.c -o p21
mohon@abc123:~/temp$ ./p21 4
Fibonacci sequence up to 4 terms:
0 1 1 2
mohon@abc123:~/temp$ ./p21 6
Fibonacci sequence up to 6 terms:
0 1 1 2 3 5
mohon@abc123:~/temp$ ./p21 10
Fibonacci sequence up to 10 terms:
0 1 1 2 3 5 8 13 21 34
mohon@abc123:~/temp$ ./p21 8
Fibonacci sequence up to 8 terms:
0 1 1 2 3 5 8 13
mohon@abc123:~/temp$ ./p21 20
Fibonacci sequence up to 20 terms:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
mohon@abc123:~/temp$
```

Answer - 22

Problem: Implement a server-client model to provide services to client processes running in different terminals. Explain what you experience when you: ● Server process being a single threaded process tries to provide services to multiple client processes. ● Server process being a multi-threaded process tries to provide services to multiple client processes. ● Server process being a single threaded process tries to provide services to multiple client processes with multiple child processes.

Code:

Single_thread_server.c

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>

int main() {
    int server_fd, client_fd;
    struct sockaddr_in addr;
```

```
    char buffer[100];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(1234);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("Server running...\n");

    while (1) {
        client_fd = accept(server_fd, NULL, NULL);
        read(client_fd, buffer, sizeof(buffer));
        printf("Client says: %s\n", buffer);
        write(client_fd, "Hi from server\n", 15);
        sleep(2); // Simulate some processing time
        close(client_fd);
    }

    return 0;
}
```

[2], [5], [3] [4]

Multithread_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <netinet/in.h>

void* handle_client(void* arg) {
    int client_fd = *(int*)arg;
    char buffer[100];

    read(client_fd, buffer, sizeof(buffer));
    printf("Client says: %s\n", buffer);
```

```
    write(client_fd, "Hi from multi-threaded server\n", 31);
    close(client_fd);

    free(arg); // Free dynamically allocated memory
    return NULL;
}

int main() {
    int server_fd;
    struct sockaddr_in addr;
    socklen_t addr_len = sizeof(addr);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(1234);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("Multi-threaded server running...\n");

    while (1) {
        int* client_fd = malloc(sizeof(int));
        *client_fd = accept(server_fd, (struct sockaddr*)&addr,
&addr_len);

        pthread_t tid;
        pthread_create(&tid, NULL, handle_client, client_fd);
        pthread_detach(tid); // Automatically clean up finished threads
    }

    return 0;
}
```

[2], [5], [3]

Multiprocess_server:

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <sys/wait.h>
#include <signal.h>

void handle_client(int client_fd) {
    char buffer[100];

    read(client_fd, buffer, sizeof(buffer));
    printf("Client says: %s\n", buffer);
    write(client_fd, "Hello from server (child process)\n", 35);
    close(client_fd);
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in addr;
    socklen_t addr_len = sizeof(addr);

    signal(SIGCHLD, SIG_IGN);  // Prevent zombie processes

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(1234);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("Server is running...\n");

    while (1) {
        client_fd = accept(server_fd, (struct sockaddr*)&addr, &addr_len);

        if (fork() == 0) {
            // Child process
            close(server_fd);  // Child doesn't need listening socket
            handle_client(client_fd);
```

```
            exit(0);  // End child
        } else {
            // Parent process
            close(client_fd);  // Parent doesn't need client socket
        }
    }

    return 0;
}
```

[2], [5], [3]

Client:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
    int sock;
    struct sockaddr_in addr;
    char buffer[100];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(1234);  // Server port
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(sock, (struct sockaddr*)&addr, sizeof(addr));

    write(sock, "Hello Server\n", 13);
    read(sock, buffer, sizeof(buffer));
    printf("Server replied: %s\n", buffer);

    close(sock);
    return 0;
}
```

[1] , [2], [5], [3]

Output:
Single thread:

```
mohon@abc123:~/temp$ ./p122
Server running...
Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server
```

```
_client
mohon@abc123:~/temp$ ./p22_client
mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$
mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server
```

```
mohon@abc123:~/temp$ ./p22_client
mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from server
```

Case A: Single-threaded Server

- The server handles one client at a time.
- Other clients must wait if one is already being served.
- Result: Slow performance and delays when many clients connect.

Multithread_server:

- The server creates a new thread for each client.
- Multiple clients are served in parallel.
- Result: Fast and efficient, good for real-time service

```
mohon@abc123:~/temp$ ./p222
Multi-threaded server running...
Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server
```

```
mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded serve
r
```

```
mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$ ./p22_client
Server replied: Hi from multi-threaded server

mohon@abc123:~/temp$
```

Multiprocess_server:

- The server creates a new process for each client.
- Each client is handled separately, but using more memory than threads.
- Result: Good isolation, but slower than threads due to process overhead.

```
mohon@abc123:~/temp$ ./p322
Server is running...
Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server

Client says: Hello Server
```

```
mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child pro
cess)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child pro
cess)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child pro
cess)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child pro
cess)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child pro
cess)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child pro
cess)
```

```
mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$ ./p22_client
Server replied: Hello from server (child process)

mohon@abc123:~/temp$
```

References:

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

[2] Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). *Operating Systems: Three Easy Pieces* (Version 1.10). Arpaci-Dusseau Books. Retrieved from

[3] OpenAI. (2023). *ChatGPT* (Mar 14 version) [Large language model].

[4] Microsoft. (2025). *Copilot (GPT-4)* [Large language model].

[5] Class-concept